

GymPro V2

Technical Documentation

Database Design, System Architecture,
Code Implementation and Verification

Project type	University gym-management system
Architecture	Server-rendered PHP with service layer
Database	10 business tables + migration metadata
Roles	Administrator, Receptionist, Trainer, Member
Document version	V2 technical reference • September 2026

Table of Contents

- 01** Introduction and Scope
- 02** Requirements, Roles and Technology
- 03** System Architecture
- 04** Database Design and Relationships
- 05** Table-by-Table Explanation
- 06** Database Integrity Decisions
- 07** Bootstrap and Shared Infrastructure
- 08** Authentication and Authorization
- 09** Membership and Payment Implementation
- 10** Class and Enrollment Implementation
- 11** Attendance and Gym Visit Implementation
- 12** Role-Specific Web Pages
- 13** Migration, Demo Data and Testing
- 14** Security Controls
- 15** Limitations and Future Work
- 16** Installation and Verification
- 17** Traceability and Source Appendix

1. Introduction and Scope

This chapter establishes the purpose, educational context, and intentionally focused boundary of GymPro V2.

GymPro V2 is a role-based gym-management information system implemented as an isolated application inside the repository's `v2/` directory. It demonstrates how a normalized relational database, transactional PHP services, server-rendered pages, and security-focused shared helpers can support common gym workflows.

Project objectives

- Maintain secure accounts and separate workspaces for administrators, receptionists, trainers, and members.
- Represent member and trainer profiles without duplicating authentication credentials.
- Track membership plans, purchased memberships, and simulated payment records.
- Support class creation, administrative review, member requests, trainer decisions, and attendance.
- Record physical gym entry and exit independently from class attendance.
- Demonstrate database integrity through keys, constraints, transactions, checksums, and contract tests.

Scope boundary

V2 contains exactly ten business tables. `SCHEMA_MIGRATION` is an additional infrastructure table and is not counted as a business entity.

Intentionally excluded

Messaging, notifications, trainer qualification-document uploads, commissions and payouts, audit-log tables, scheduled-job infrastructure, and password-reset/email-verification token tables are outside the V2 scope. The payment workflow is a local simulation rather than a connection to a financial provider.

2. Requirements, Roles and Technology

The application uses a small conventional stack that can be installed through XAMPP and explained clearly in an academic setting.

Component	Selection	Purpose
Server language	PHP 8.1+	Strictly typed application, services, pages and CLI tools
Database	MariaDB 10.5+ / MySQL 8.0+	InnoDB transactions, constraints and utf8mb4 storage
Data access	mysqli	Prepared statements and explicit transactions
Presentation	HTML + CSS	Responsive server-rendered interface
Enhancement	Vanilla JavaScript	Navigation, alerts, loading state and password visibility
Local environment	XAMPP on Windows	Apache, PHP and MariaDB development runtime

Role responsibility matrix

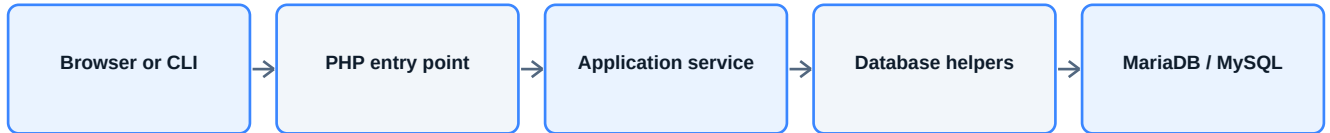
Role	Primary capabilities	Workspace
ADMIN	Dashboard reporting; member, plan, trainer, payment and staff visibility; draft-class review	admin/
RECEPTIONIST	Member facility check-in/check-out and visit-history review	reception/
TRAINER	Create classes, decide pending enrollments and mark assigned-class attendance	trainer/
MEMBER	Register, select membership, record simulated payment, request classes and view history	member/

Authorization model

Each role is guarded separately. An administrator is not automatically granted receptionist or trainer routes.

3. System Architecture

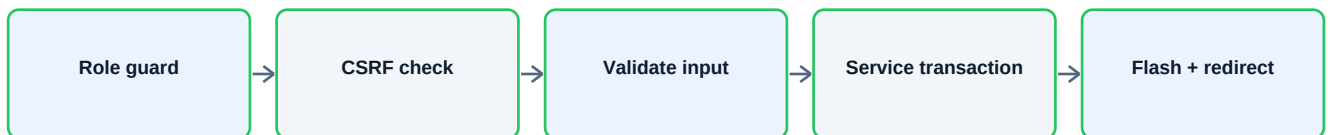
GymPro V2 separates HTTP/CLI entry points, domain services, shared infrastructure, and persistence responsibilities.



Layer responsibilities

Layer	Representative files	Responsibility
Entry points	auth/, admin/, member/, trainer/, reception/, cli/	Receive requests, invoke guards/services, render or redirect
Application services	app/Services/*.php	Authorization-aware business rules and lifecycle transitions
Shared infrastructure	bootstrap/app.php, bootstrap/helpers.php, config/	Environment, autoloading, sessions, CSRF, database and transactions
Persistence	database/migrations/001_create_core_schema.sql	Durable entities, keys, checks, indexes and relationships

Typical state-changing request



Pages do not concatenate submitted values into SQL. Values are bound through prepared statements. Multi-record transitions, such as inserting a payment and activating its membership, are wrapped in **transaction()** so either all changes commit or all are rolled back.

4. Database Design and Relationships

The relational model connects identity, membership, scheduling, participation, and facility-access data while retaining a compact ten-table boundary.

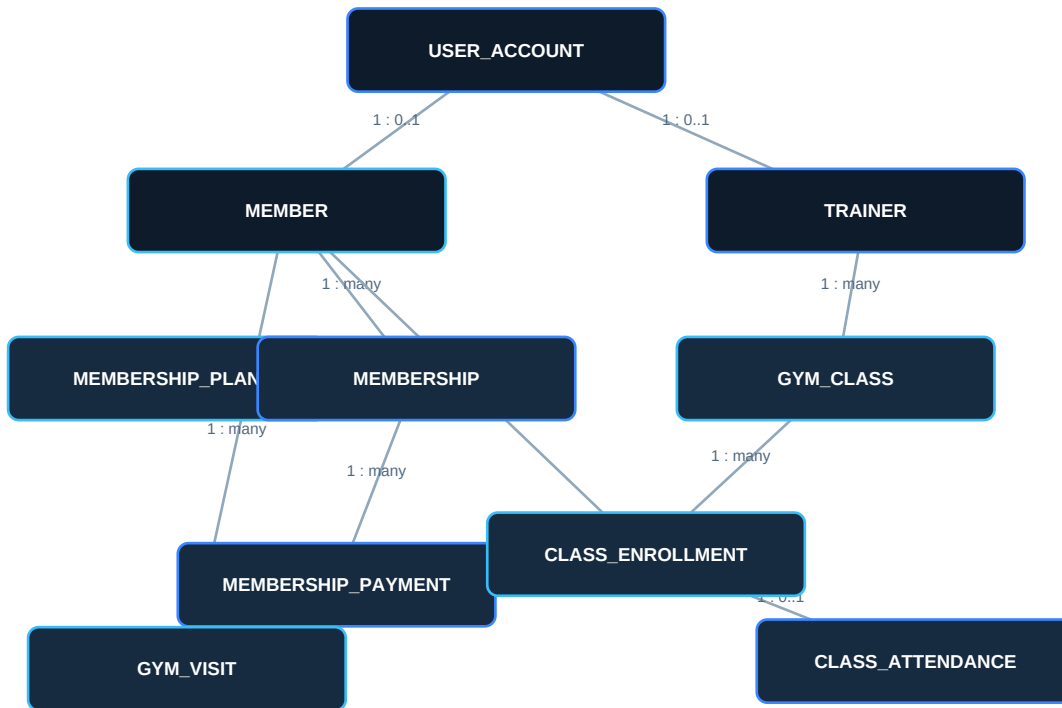


Figure 1. Simplified entity-relationship overview. Actor references from `USER_ACCOUNT` to payment, attendance and visit records are omitted from some lines for visual clarity.

Domain grouping

Domain	Tables	Purpose
Identity	USER_ACCOUNT, MEMBER, TRAINER	Credentials, roles and one-to-one profile types
Membership	MEMBERSHIP_PLAN, MEMBERSHIP, MEMBERSHIP_PAYMENT	Catalog terms, purchased periods and financial records
Scheduling	GYM_CLASS	Trainer-led sessions, timing, location, capacity and lifecycle
Participation	CLASS_ENROLLMENT, CLASS_ATTENDANCE	Member requests, decisions and class participation
Facility access	GYM_VISIT	Reception-controlled physical entry and exit

Relationship summary

Parent	Child	Cardinality	Meaning
USER_ACCOUNT	MEMBER / TRAINER	1 to 0..1	An account may own one role-appropriate profile
MEMBERSHIP_PLAN	MEMBERSHIP	1 to many	A catalog plan can be purchased repeatedly
MEMBER	MEMBERSHIP	1 to many	Membership history is retained
MEMBERSHIP	MEMBERSHIP_PAYMENT	1 to many	Attempts or lifecycle payments belong to a membership

Parent	Child	Cardinality	Meaning
TRAINER	GYM_CLASS	1 to many	A trainer teaches classes
MEMBER + GYM_CLASS	CLASS_ENROLLMENT	many to many	Enrollment is the association entity
CLASS_ENROLLMENT	CLASS_ATTENDANCE	1 to 0..1	One final row per enrollment
MEMBER	GYM_VISIT	1 to many	A member accumulates visit history

5. Table-by-Table Explanation

Each table below is described in terms of its business purpose, identity, relationships, lifecycle and important safeguards.

5.1 USER_ACCOUNT

Central authentication identity for every user. Credentials and role are stored once, while member/trainer details live in role-specific profile tables.

Aspect	Explanation
Primary identity	UserAccountID (auto-increment); Email is unique.
Relationships	Optional one-to-one MEMBER or TRAINER profile; actor references from payments, attendance and visits.

- Role is restricted to ADMIN, RECEPTIONIST, TRAINER or MEMBER.
- PasswordHash stores password_hash() output rather than plaintext.
- FailedLoginCount and LockedUntil support temporary account lockout.
- Status allows pending, active, suspended and disabled accounts without deleting history.

5.2 MEMBER

Stores the member profile, public number, contact details, emergency contact and lifecycle state.

Aspect	Explanation
Primary identity	MemberID; MemberNumber and UserAccountID are independently unique.
Relationships	Belongs to one MEMBER account; owns memberships, class enrollments and gym visits.

- Separating the profile from USER_ACCOUNT keeps authentication fields normalized.
- JoinedOn records the business join date; Status supports non-destructive deactivation.
- Foreign-key deletion is restrictive to protect operational history.

5.3 TRAINER

Stores the trainer's public identity, specialization, biography, hire date and profile lifecycle.

Aspect	Explanation
Primary identity	TrainerID; TrainerNumber and UserAccountID are independently unique.
Relationships	Belongs to one TRAINER account; may be assigned to many gym classes.

- Profile status is checked in addition to account status for trainer operations.
- TrainerID becomes NULL on a class if deletion were permitted, preserving class history.

5.4 MEMBERSHIP_PLAN

Defines reusable catalog terms offered to members.

Aspect	Explanation
Primary identity	MembershipPlanID; Name is unique.
Relationships	Referenced by many MEMBERSHIP records.

- DurationMonths must be positive and Price must be non-negative.
- Currency is constrained to uppercase; ClassLimit is nullable for unlimited access.
- IsActive controls availability for new selection without removing historical references.

5.5 MEMBERSHIP

Represents one member's purchased entitlement over an inclusive date interval.

Aspect	Explanation
Primary identity	MembershipID.
Relationships	Belongs to MEMBER and MEMBERSHIP_PLAN; receives MEMBERSHIP_PAYMENT rows.

- PlanNameSnapshot, PriceSnapshot and CurrencySnapshot preserve purchase-time terms even if the plan changes later.
- EndsOn must be on or after StartsOn.
- Lifecycle values are PENDING, ACTIVE, PAUSED, CANCELLED and EXPIRED.

5.6 MEMBERSHIP_PAYMENT

Stores a payment attempt/result associated with a membership.

Aspect	Explanation
Primary identity	MembershipPaymentID; transaction reference and idempotency key are nullable unique identifiers.
Relationships	Belongs to MEMBERSHIP; optionally records the acting USER_ACCOUNT.

- Amount must be positive and currency uppercase.
- A PAID row must have PaidAt.
- The idempotency key lets a repeated request return the existing payment instead of inserting another.
- No card number, CVV or expiry value is stored.

5.7 GYM_CLASS

Represents a trainer-led scheduled session with timing, capacity, venue and lifecycle.

Aspect	Explanation
Primary identity	GymClassID.
Relationships	Optionally belongs to TRAINER; owns CLASS_ENROLLMENT rows.

- EndsAt must be later than StartsAt and Capacity must be positive.
- Statuses are DRAFT, SCHEDULED, CANCELLED and COMPLETED.
- Application logic checks trainer schedule overlap before insertion or approval.

5.8 CLASS_ENROLLMENT

Association and workflow record connecting one member to one class.

Aspect	Explanation
Primary identity	ClassEnrollmentID; (GymClassID, MemberID) is unique.
Relationships	Belongs to GYM_CLASS and MEMBER; may own one CLASS_ATTENDANCE row.

- Statuses are WAITLISTED, ENROLLED and CANCELLED.
- The unique class/member pair prevents duplicate requests.
- CancelledAt is required whenever the row is CANCELLED.

5.9 CLASS_ATTENDANCE

Stores the attendance outcome for an enrolled member in a specific class.

Aspect	Explanation
Primary identity	ClassAttendanceID; ClassEnrollmentID is unique.
Relationships	Composite reference to CLASS_ENROLLMENT(ClassEnrollmentID, GymClassID); marked by USER_ACCOUNT.

- Statuses are PRESENT, ABSENT, LATE and EXCUSED.
- PRESENT and LATE require CheckedInAt.
- The composite relationship prevents an attendance row from naming a class different from its enrollment.

5.10 GYM_VISIT

Records physical facility entry and exit, independently of participation in a class.

Aspect	Explanation
Primary identity	GymVisitID; generated OpenVisitMarker participates in a unique key.
Relationships	Belongs to MEMBER; check-in and checkout actors reference USER_ACCOUNT.

- Checkout cannot occur before check-in and requires a checkout actor.
- OpenVisitMarker is 1 only while CheckedOutAt is NULL; uniqueness with MemberID permits one open visit per member.
- Completed visits retain both timestamps and reception actors for operational traceability.

5.11 SCHEMA_MIGRATION (infrastructure)

This table is not a domain entity. It stores each migration identifier, SHA-256 checksum, start/completion timestamps, elapsed milliseconds and dirty state. The migrator uses it to reject modified applied migrations and detect interrupted work.

6. Database Integrity Decisions

Important rules are enforced at the lowest practical level: declaratively in the database where possible and transactionally in services where the rule depends on current business state.

Business rule	Enforcement	Mechanism
One login per email	Database	Unique Email key
One profile per account/type	Database	Unique UserAccountID in profile
One request per member/class	Database	Unique (GymClassID, MemberID)
Attendance belongs to exact class	Database	Composite foreign key
One open visit per member	Database	Generated marker + unique key
Payment retry does not duplicate	Database + service	Unique idempotency key + locked lookup
Membership covers class date	Service	Date-bounded ACTIVE membership query
Trainer schedule does not overlap	Service transaction	Actor lock followed by overlap query
Class capacity is available	Service transaction	Locked class plus enrolled count
Attendance matches roster	Service	Exact sorted enrollment-ID comparison

Purchase snapshots



Composite attendance identity

```

FOREIGN KEY (ClassEnrollmentID, GymClassID)
REFERENCES CLASS_ENROLLMENT (ClassEnrollmentID, GymClassID)

```

Repeating GymClassID in CLASS_ATTENDANCE supports direct class reporting. The composite key ensures this denormalized convenience cannot contradict the enrollment's class.

One-open-visit invariant

```

OpenVisitMarker = CASE WHEN CheckedOutAt IS NULL THEN 1 ELSE NULL END
UNIQUE (MemberID, OpenVisitMarker)

```

MySQL/MariaDB unique keys allow multiple NULL values. Completed visits therefore coexist, while a second row with the same MemberID and marker 1 is rejected.

7. Bootstrap and Shared Infrastructure

The lightweight bootstrap replaces a large framework while retaining centralized configuration, sessions, autoloading and database primitives.

Function / component	Implementation purpose
bootstrap/app.php	Defines V2_ROOT, loads helpers and .env, registers GymProV2 autoloading, starts secure sessions and sends headers
env(), env_bool(), env_int()	Read and validate environment configuration
config()	Loads nested values from config/*.php with static caching
base_url(), redirect()	Build application URLs and perform HTTP redirects
csrf_token(), csrf_field(), verify_csrf()	Create session token, render hidden field and reject invalid mutations
db(), db_all(), db_one(), db_execute()	Open strict mysqli connection and execute prepared queries
transaction()	Begin, commit and rollback multi-query operations
current_user(), require_*	Resolve active sessions and enforce guest/login/role boundaries
flash(), flash_input()	Carry messages and save old form values across redirects
e()	Escape untrusted text for HTML output

Session and response protections

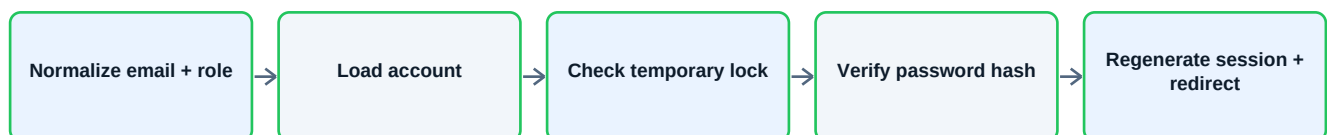
- Strict session mode and cookie-only sessions.
- HttpOnly and SameSite cookies; optional Secure cookies for HTTPS.
- Session ID regeneration after successful login.
- X-Content-Type-Options, X-Frame-Options, Referrer-Policy and Permissions-Policy headers.
- Existing sessions are revoked when the account is no longer ACTIVE.

8. Authentication and Authorization

AuthService implements member/trainer registration, role-aware login, lockout accounting, session establishment and logout.

Method	Primary behavior	Tables
registerMember(data)	Validate fields; atomically create ACTIVE member account and profile	USER_ACCOUNT, MEMBER
registerTrainer(data)	Validate fields; atomically create PENDING trainer account and profile	USER_ACCOUNT, TRAINER
login(email, password, expectedRole)	Find role account, check lock, verify hash, reset failures and create session	USER_ACCOUNT
logout()	Clear server session and expire session cookie	—

Login flow



For an unknown account, a dummy password hash is still verified to reduce obvious timing differences. Passwords are created with `password_hash()` and checked with `password_verify()`. The selected portal role is matched against the account role.

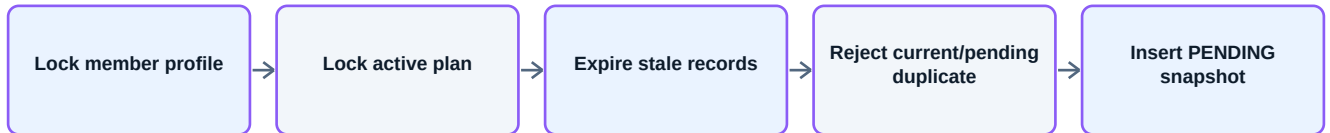
Current activation limitation

Trainer registration creates USER_ACCOUNT and TRAINER rows as PENDING, but the current administrator trainer page has no activation action. Activation therefore requires a future workflow rather than being fully usable through the V2 browser interface.

9. Membership and Payment Implementation

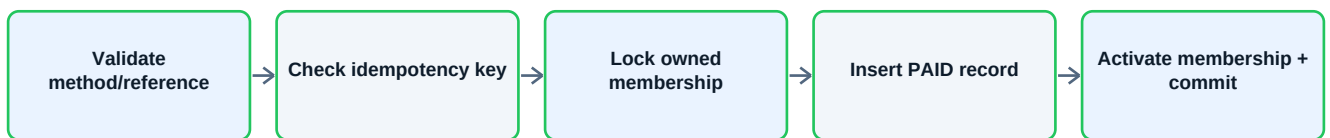
MembershipService controls plan selection, current membership lookup and atomic simulated payment activation.

Selecting a plan



The service computes an inclusive end date by adding the selected DurationMonths and subtracting one day. It copies plan name, price and currency into the membership so historical terms survive catalog changes.

Recording a simulated payment



The transaction binds the payment to a membership owned by the acting member. An existing idempotency key returns its prior row; a successful insert and membership activation commit together.

Demonstration payment only

No payment gateway or trusted external confirmation is integrated. Current member-submitted form data directly records PAID and activates membership. This is suitable for local workflow demonstration, not real financial settlement.

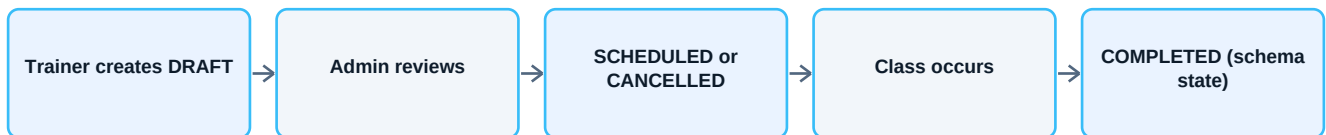
Data minimization

The application never requests or stores full card numbers, CVV values or card expiry dates.

10. Class and Enrollment Implementation

ClassService owns future class creation, administrative draft review, member enrollment requests and trainer decisions.

Class lifecycle



Creation validates name, description, location, capacity, start/end ordering and future timing. Locking the trainer actor serializes overlap checks for that trainer. Administrative approval verifies the class is still DRAFT, future-dated, assigned to an active trainer and not overlapping.

Enrollment lifecycle



A member needs an ACTIVE membership whose date range covers the class. The trainer may decide only requests for their own future SCHEDULED class. Before approval, the service checks membership entitlement, class-limit consumption and enrolled capacity.

Current lifecycle gaps

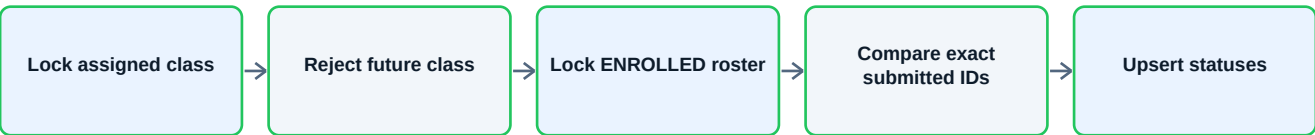
Members cannot withdraw WAITLISTED or ENROLLED rows, trainers cannot cancel an already ENROLLED member, and no application action currently transitions a SCHEDULED class to COMPLETED. Review notes and enrollment reasons are accepted but not persisted by the ten-table schema.

11. Attendance and Gym Visit Implementation

AttendanceService deliberately separates participation in a class from physical access to the gym facility.

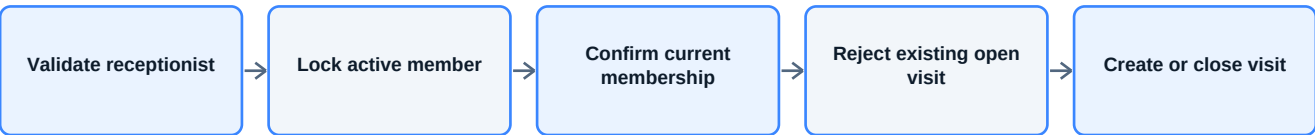
Concept	CLASS_ATTENDANCE	GYM_VISIT
Purpose	Participation in an organized class	Physical entry and exit
Member link	Through CLASS_ENROLLMENT	Direct MEMBER foreign key
Primary actor	Assigned TRAINER account	RECEPTIONIST account
Time data	Class check-in where relevant	Facility CheckedInAt / CheckedOutAt
Duplicate guard	One row per enrollment	One open visit per member

Class attendance algorithm



Only the assigned active trainer can submit. The submitted key set must exactly equal the sorted enrolled roster, preventing omitted or injected enrollment IDs. PRESENT and LATE receive a check-in timestamp; ABSENT and EXCUSED do not.

Reception check-in/out



Attendance history limitation

Resubmission updates the existing attendance row, including for COMPLETED classes. No correction deadline or historical revision record is retained, so the current implementation should not be described as an immutable audit trail.

12. Role-Specific Web Pages

Thin PHP controllers use shared guards, services and templates to expose each role's workflow.

Area	Route	Behavior
Public	index.php	Landing page and links to authentication
Authentication	auth/login.php	Role selection, credential submission and dashboard redirect
Authentication	auth/register-member.php	Self-service member registration
Authentication	auth/register-trainer.php	Pending trainer application
Member	member/index.php	Membership, class and attendance summary
Member	member/membership.php	Current membership and plan selection
Member	member/checkout.php	Simulated payment record submission
Member	member/classes.php	Browse scheduled classes and request enrollment
Member	member/attendance.php	Class participation history
Trainer	trainer/classes.php	Create and list own classes
Trainer	trainer/enrollments.php	Approve/cancel waitlisted requests
Trainer	trainer/attendance.php	Mark exact enrolled roster
Reception	reception/index.php	Check members in/out
Reception	reception/history.php	Search visit history by member/date
Admin	admin/index.php	Operational dashboard statistics
Admin	admin/classes.php	Review DRAFT classes
Admin	admin/members.php, plans.php, trainers.php	Directories and reporting views
Admin	admin/payments.php, staff.php	Payment ledger and staff-account reporting

The shared `includes/header.php` builds role-specific navigation, while `includes/footer.php` loads the local JavaScript. `assets/css/app.css` provides the responsive application shell, forms, cards, alerts and tables. `assets/js/app.js` progressively enhances mobile navigation, alert dismissal, submission feedback and password visibility.

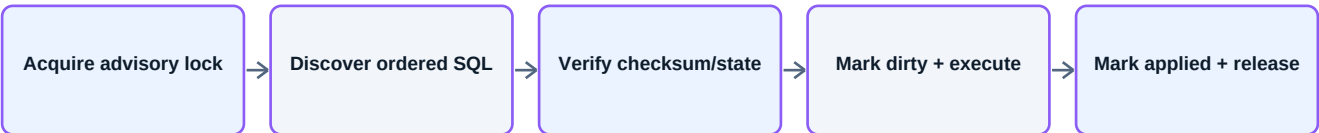
Administrative scope

Several administrator pages are read-only reporting directories rather than full create/read/update/delete modules. In particular, plan creation/editing and trainer activation are not currently exposed.

13. Migration, Demo Data and Testing

CLI tools make schema setup repeatable, protect migration history, and populate safe fictional demonstrations.

Migration runner



cli/migrate.php supports migration, status and dry-run modes. It stores SHA-256 checksums, refuses modified applied files, detects dirty records and prevents concurrent migration processes with a database advisory lock.

Demo seeder

- Runs only from CLI and refuses execution unless APP_ENV=local.
- Uses clearly fictional example.test accounts and the documented local password Demo@12345.
- Creates one starter scenario and five connected scenarios numbered 101 through 105.
- Exercises every business table while preserving the four canonical role values.
- Upserts only reserved markers inside one transaction; it never truncates unrelated data.

Schema contract

tests/schema_contract.php statically parses the migration and verifies the exact table contract, required columns, keys, foreign keys, InnoDB and utf8mb4 declarations. When configured, it also queries INFORMATION_SCHEMA and can require live database validation.

Verification type	Coverage	Boundary
Static schema contract	Migration structure and declared constraints	Does not execute browser workflows
Optional live metadata	Actual table/column/constraint metadata	Requires reachable configured database
Manual workflow checklist	Roles, CSRF, payment, classes, attendance and visits	Not a broad automated suite
Seeder rerun	Reserved-row idempotency and relationship integrity	Local fictional data only

14. Security Controls

V2 combines application-layer protections with database constraints, while avoiding claims of formal certification or complete production hardening.

Risk	Implemented control	Primary location
Credential disclosure	<code>password_hash()</code> / <code>password_verify()</code> ; no plaintext storage	<code>AuthService.php</code>
SQL injection	Prepared <code>mysqli</code> statements with bound values	<code>helpers.php</code> and services
Cross-site request forgery	Session token + <code>hash_equals()</code> on mutations	<code>helpers.php</code> and forms
Cross-site scripting	Contextual HTML escaping through <code>e()</code>	<code>helpers.php</code> and templates
Session fixation	Regenerate session identifier after login	<code>AuthService.php</code>
Stale account access	<code>current_user()</code> rechecks <code>ACTIVE</code> state	<code>helpers.php</code>
Unauthorized routes	<code>require_guest/login/role</code> guards	<code>helpers.php</code> and entry points
Repeated password guessing	Account failure count and temporary lock timestamp	<code>AuthService.php</code> / <code>USER_ACCOUNT</code>
Partial multi-row updates	Commit/rollback transaction helper	<code>helpers.php</code> and services
Duplicate financial submission	Unique idempotency key and locked prior lookup	<code>MembershipService.php</code> / schema

Deployment considerations

Production deployment should use HTTPS, `SESSION_SECURE=true`, a private `APP_KEY`, least-privileged database credentials, disabled debug output, backups, monitoring and web-server hardening. The local `.env` file is intentionally excluded from version control and from this document.

Known input/error issues

Some controllers cast attacker-controlled array values to strings, which can emit warnings. Public controllers also display raw Throwable messages, potentially exposing database details. These are documented defects, not security guarantees.

15. Limitations and Future Work

Transparent evaluation separates the implemented educational system from features required by a mature production service.

Area	Current limitation / recommended improvement
Payment trust	Member-controlled input immediately records PAID; integrate a trusted provider callback or staff-confirmed workflow.
Trainer activation	Add an administrator action that atomically activates both account and trainer profile.
Plan administration	Add validated administrator creation, editing and activation/deactivation operations.
Membership expiry	Centralize expiry normalization so <code>getCurrent()</code> never displays an expired ACTIVE/PAUSED row.
Attendance history	Introduce bounded corrections with retained reason/history within the chosen schema strategy.
Enrollment lifecycle	Allow member withdrawal and trainer cancellation of accepted enrollments.
Class lifecycle	Implement SCHEDULED to COMPLETED transition.
Lockout threshold	Correct assignment ordering that currently locks on the fourth failure rather than the intended fifth.
Input handling	Reject non-scalar request values before casting.
Error handling	Expose safe domain messages and log internal exceptions privately.
Automated coverage	Add service, integration, concurrency and browser tests plus CI.
Production services	Add mail/recovery, monitoring, backup and operational procedures only if scope expands.

Messaging, notifications, trainer qualification uploads, commissions/payouts, audit-log tables, scheduled jobs and reset/verification-token infrastructure remain intentionally excluded unless a future project revision changes the ten-table scope.

16. Installation and Verification

The following sequence reproduces the V2 database and documentation in a standard local XAMPP environment.

Application setup

1. Start Apache and MySQL from the XAMPP Control Panel.
2. Copy `v2/.env.example` to `v2/.env` and set local database values.
3. Create an empty utf8mb4 database named `gym_system_v2`.
4. Run the migration command and inspect status.
5. Optionally run the local-only fictional data seeder.
6. Open the `APP_URL` value in a browser.

```
C:\xampp\php\php.exe v2\cli\migrate.php
C:\xampp\php\php.exe v2\cli\migrate.php status
C:\xampp\php\php.exe v2\cli\seed-demo.php
C:\xampp\php\php.exe v2\tests\schema_contract.php
```

Regenerating this PDF

```
python -m pip install -r v2/docs/requirements-pdf.txt
python v2/docs/generate_v2_documentation.py
```

The generated file is `v2/docs/GymPro_V2_Technical_Documentation.pdf`. The generator resolves paths relative to itself and does not read `.env` or require a running web/database server.

17. Traceability and Source Appendix

This matrix connects user-facing workflows to their entry points, services and principal persistent entities.

Workflow	Page / command	Service	Principal tables
Member registration	auth/register-member.php	AuthService	USER_ACCOUNT, MEMBER
Trainer application	auth/register-trainer.php	AuthService	USER_ACCOUNT, TRAINER
Login/logout	auth/login.php, logout.php	AuthService	USER_ACCOUNT
Choose plan	member/membership.php	MembershipService	MEMBERSHIP_PLAN, MEMBERSHIP
Record payment	member/checkout.php	MembershipService	MEMBERSHIP, MEMBERSHIP_PAYMENT
Create/review class	trainer/classes.php, admin/classes.php	ClassService	TRAINER, GYM_CLASS
Request/decide enrollment	member/classes.php, trainer/enrollments.php	ClassService	MEMBERSHIP, GYM_CLASS, CLASS_ENROLLMENT
Mark class attendance	trainer/attendance.php	AttendanceService	GYM_CLASS, CLASS_ENROLLMENT, CLASS_ATTENDANCE
Facility check-in/out	reception/index.php	AttendanceService	MEMBER, MEMBERSHIP, GYM_VISIT
Install schema	cli/migrate.php	Migration runner	SCHEMA_MIGRATION + business tables

Authoritative source files

- database/migrations/001_create_core_schema.sql — canonical schema, statuses and constraints
- app/Services/AuthService.php — registration, authentication and logout
- app/Services/MembershipService.php — plan selection and simulated payment activation
- app/Services/ClassService.php — class and enrollment workflows
- app/Services/AttendanceService.php — class attendance and facility visits
- bootstrap/app.php and bootstrap/helpers.php — runtime, sessions, CSRF, database and role guards
- admin/, member/, trainer/, reception/ and auth/ — HTTP controllers and views
- cli/migrate.php and cli/seed-demo.php — schema lifecycle and fictional data
- tests/schema_contract.php — static and optional live schema verification
- README.md and docs/*.md — setup, architecture, data dictionary, ERD, workflows and testing notes

Conclusion

GymPro V2 is a focused university-project implementation with a strong normalized schema, meaningful relational constraints, transactional services and clear role boundaries. Its limitations are documented so the system can be evaluated accurately and improved systematically.